	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch: 15
	Name: A.Venu Varshith Reg. No: 2503A52447
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 06/10/2026

QUESTION 1:

TASK 1 – Improve Readability

ORIGINAL CODE:

```
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
    print("pass")
else:
    print("fail")
```

AI PROMPT;

Refactor the following Python code using meaningful variable names and create a calculate_average() function.
Improve the formatting, indentation, and overall readability without changing the program's output.
Run both the original and refactored code and verify that they produce the same result.
Also explain the improvements made in the refactored version.

AI SUGGESTION:

Replace a, b, and c with meaningful names such as mark1, mark2, and mark3.

Create a reusable calculate_average() function.

Use proper spacing and indentation.

Replace x with the meaningful name average.

Keep the original pass/fail logic unchanged.

REFACTORED CODE:

```python
def calculate_average(first_score, second_score, third_score):
    """Return the arithmetic mean of three scores."""
    return (first_score + second_score + third_score) / 3


first_score = 80
second_score = 70
third_score = 90

average_score = calculate_average(first_score, second_score, third_score)

if average_score >= 50:
    print("pass")
else:
    print("fail")
```

OUTPUT:

Original Code:

pass

Refactored Code:

pass

EXPLAINATION:

The legacy code was refactored using meaningful variable names and a reusable calculate_average() function. The formatting and indentation were also improved, making the program easier to read and understand.

Both the original and refactored versions produced the same output, **“pass”**, so the original behavior was preserved. The main improvement is that the refactored code is clearer, more organized, reusable, and easier to maintain.

QUESTION 2:

TASK 2 – Remove Repeated Code

ORIGINAL CODE:

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")
name2 = "Priya"
marks2 = 72
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

AI PROMPT:

Identify the duplicated logic in the following Python code and refactor it into a reusable function.
Test the function with at least four students and make sure the output remains correct.
Improve the code's readability and maintainability without changing its functionality.
Explain how removing code duplication makes the program easier to maintain.

AI SUGGESTION:

The if-else logic for checking whether a student passes or fails is repeated. Create a reusable function called check_result() that accepts the student's name and marks, then prints the appropriate result.

REFACTORED CODE:

```python
def print_result(name, marks):
    if marks >= 50:
        print(name, "Pass")
    else:
        print(name, "Fail")


students = [
    ("Arun", 85),
    ("Priya", 72),
    ("Rahul", 45),
    ("Ananya", 50),
]

for name, marks in students:
    print_result(name, marks)
```

OUTPUT:

```
Arun Pass
Priya Pass
Rahul Fail
Ananya Pass
```

EXPLAINATION:

The results are correct because students with marks **50 or above** pass, while students below 50 fail.

The repeated if-else logic was moved into the reusable check_result() function. Now, instead of writing the same code for every student, we only call the function with the student's name and marks.

Removing duplication improves **maintainability** because if the pass condition changes in the future, we only need to modify the function once instead of changing the same logic in multiple places. It also makes the code shorter, cleaner, and easier to understand.

QUESTION 3:

TASK 3 – Refactor a Long Conditional

ORIGINAL CODE:

age = 68
temperature = 39

```
if age >= 60 and temperature >= 38:
    print("High Priority")
elif age >= 60 and temperature < 38:
    print("Medium Priority")
elif age < 60 and temperature >= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

AI PROMPT:

Simplify the conditional logic in the following Python code and create a function called determine_priority().
Test the function with at least four different age and temperature combinations.
Compare the original and refactored logic and verify that the output remains the same.
Explain whether the refactored code preserves the original behavior.

AI SUGGESTION:

The three conditions can be simplified because **High Priority** occurs only when both age is 60 or above and temperature is 38 or above. In all other cases where only one condition is true, the result is **Medium Priority**.

REFACTORED CODE:

```python
def original_determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 and temperature < 38:
        return "Medium Priority"
    elif age < 60 and temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"


def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 or temperature >= 38:
        return "Medium Priority"
    return "Normal Priority"


if __name__ == "__main__":
    test_cases = [
        (68, 39),
        (68, 37),
        (59, 39),
        (59, 37),
    ]

    for age, temperature in test_cases:
        original = original_determine_priority(age, temperature)
        refactored = determine_priority(age, temperature)
        print(
            f"Age {age}, temperature {temperature}: "
            f"original={original}, refactored={refactored}"
        )
        assert original == refactored

    print("All tests passed; the refactored logic preserves the original behavior.")
```

OUTPUT:

```
Age 68, temperature 39: original=High Priority, refactored=High Priority
Age 68, temperature 37: original=Medium Priority, refactored=Medium Priority
Age 59, temperature 39: original=Medium Priority, refactored=Medium Priority
Age 59, temperature 37: original=Normal Priority, refactored=Normal Priority
All tests passed; the refactored logic preserves the original behavior.
```

EXPLAINATION:

The original code contained multiple conditions that repeated the same age and temperature checks. The logic was simplified by using the or operator inside the determine_priority() function, making the code shorter and easier to understand.

The refactored function was tested with four different age and temperature combinations. In every case, the refactored code produced the same result as the original code: **High Priority, Medium Priority, or Normal Priority**.

Therefore, the AI successfully **preserved the original behavior** while improving readability, reducing repeated conditions, and making the code easier to maintain and reuse.

QUESTION 4:

TASK 4 – Refactor Inefficient List Processing

ORIGINAL CODE:

```
patient_ids = [101, 205, 309, 412, 518, 623]

target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

AI PROMPT:

Improve the readability and efficiency of the following Python code for patient ID lookups.
Explain whether a set or dictionary is better for frequent lookups and justify your choice.
Refactor the code using the suggested data structure and test it with both existing and missing patient IDs.
Compare the typical lookup complexity of a list and a set.

AI SUGGESTION:

A **set** is better for frequent patient ID lookups because it provides average **O(1)** membership checking. A list requires **O(n)** time for a typical lookup because it may need to check each element. Since only patient IDs are being stored and searched, a set is more suitable than a dictionary.

REFACTORED CODE:

```
patient_ids = {101, 205, 309, 412, 518, 623}


def patient_exists(patient_id: int) -> bool:
    return patient_id in patient_ids


for target in (412, 999):
    if patient_exists(target):
        print(f"{target}: Patient Found")
    else:
        print(f"{target}: Patient Not Found")
```

OUTPUT:

```
412: Patient Found
999: Patient Not Found
```

EXPLAINATION:

The original code manually loops through the entire list to find a patient ID. The refactored version uses a **set** and the in operator, which makes the code shorter, clearer, and more efficient for frequent lookups.

The code was tested with both an existing ID (412) and a missing ID (700), and both produced the correct results. A list has **O(n)** typical lookup complexity, while a set has **O(1) average** lookup complexity. Therefore, using a set is a justified improvement when patient IDs need to be searched frequently.

QUESTION 5:

TASK 5 – Complete Legacy Code Review with AI

ORIGINAL CODE:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

AI PROMPT:

Review and refactor the following Python code and identify at least five improvements.
Use meaningful variable names, reusable functions, and suitable Python features to improve clarity.
Run both the original and refactored versions and verify that they produce equivalent results.
List each AI suggestion as Accepted or Rejected with a reason, and identify the most useful refactoring change.

AI SUGGESTION:

Use meaningful variable names – **Accepted:** Makes the code easier to understand.

Create a function to calculate the average – **Accepted:** Makes the calculation reusable.

Create a function to display pass/fail results – **Accepted:** Separates responsibilities.

Use sum() with a generator expression – **Accepted:** Shorter and clearer than manually adding values.

Use tuple unpacking – **Accepted:** name, marks is clearer than s[0] and s[1]. **Use a dictionary instead of tuples** – **Rejected:** Not necessary for this simple data structure and would add complexity.

REFACTORED CODE:

```python
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

PASSING_SCORE = 50


def calculate_average(student_records):
    total_score = sum(score for _, score in student_records)
    return total_score / len(student_records)


def print_student_results(student_records, passing_score=PASSING_SCORE):
    for student_name, score in student_records:
        result = "Pass" if score >= passing_score else "Fail"
        print(student_name, result)

    print(f"Average = {calculate_average(student_records)}")


if __name__ == "__main__":
    print_student_results(students)
```

OUTPUT:

```
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75
```

EXPLAINATION:

The legacy code was improved by using meaningful names, reusable functions, tuple unpacking, and Python's sum() feature. The code was divided into separate functions for calculating the average and displaying student results, making it easier to read, test, and maintain.

All important AI suggestions were reviewed and accepted except replacing tuples with dictionaries, because it was unnecessary for this simple problem. The **most useful change was creating reusable functions**, because it separates tasks and allows the same logic to be reused without rewriting code.

Both versions produced the same student results and average of **65.75**, confirming that the refactored code preserved the original behavior.